

Tab 1



Course Code: CEF 440

Course Name: Internet Programming and Mobile programming

Course Instructor: Dr Nkemeni Valery

GROUP 11

Name	Matricule
FUNWI KELSEA NDOHNWI	FE23A066
NGADANG ASSONFACK P. ESTELLE	FE23A109
ATEH SWIRRI FOFANG	FE23A018
TABOT GHISLAIN OROCK	FE23A146

Task 1: Mobile App Development Process

Introduction

Mobile applications are software programs designed to run on mobile devices such as smartphones and tablets. They are classified based on how they are built, deployed, and how they interact with the device hardware and operating system.

The mobile app development process is a step-by-step approach used to design, build, test, and launch a mobile application.

1. Idea & Planning

This is the starting point.

What happens here:

- Identify the problem the app will solve
- Define target users
- Set goals and objectives

Example: Creating a food delivery app for students

2. Requirement Gathering & Analysis

Collect and understand what the app should do.

Types of Requirements:

Functional Requirements (what the app does)

- Login system
- Add to cart

Non-functional Requirements (how the app performs)

- Speed
- Security
- Performance

3. Design (UI/UX)

Design how the app will look and behave.

Includes:

- Wireframes (basic layout)
- Mockups (visual design)
- User Experience (UX)

Tools: Figma, Adobe XD

4. Development (Coding Phase)

Actual building of the app.

Two parts:

- Frontend → what users see
- Backend → server, database, logic

Technologies:

- Android: Kotlin, Java
- iOS: Swift
- Cross-platform: Flutter, React Native

5. Testing

Check if the app works correctly.

Types of Testing:

- Functional testing
- Performance testing
- Security testing
- Usability testing

Goal: Remove bugs and errors

6. Deployment

Release the app to users.

Platforms:

- Google Play Store
- Apple App Store

7. Maintenance & Updates

After launch, the app must be maintained.

Includes:

- Fixing bugs
- Adding new features
- Improving performance

Introduction to Mobile Applications

Mobile applications, commonly called mobile apps, are software programs designed to run on mobile devices such as smartphones and tablets. These apps help users perform different tasks like communication, entertainment, banking, learning, and shopping.

Mobile apps are not all the same — they are classified based on several important factors:

1. Development Method

This refers to how the app is built.

Types:

- Native Development → Built specifically for one platform (Android or iOS)
- Web Development → Built using web technologies and runs in a browser
- Hybrid Development → Combination of web and native
- Cross-platform Development → One codebase used for multiple platforms

Why it matters:

The development method affects:

- Cost
- Speed of development
- Performance

2. Platform Compatibility

This means which devices or operating systems the app can run on.

Examples:

- Android apps → run only on Android devices
- iOS apps → run only on Apple devices
- Cross-platform apps → run on both

Why it matters:

- Determines your audience reach
- More platforms = more users but higher cost

3. Hardware Accessibility

This refers to how much of the phone's hardware the app can use.

Hardware includes:

- Camera
- GPS

- Microphone
- Bluetooth
- Storage

Comparison:

- Native apps → Full access
- Web apps → Limited access
- Hybrid/Cross-platform → Partial to near full

Why it matters:

Apps like Uber or Snapchat need strong hardware access to function properly.

4. Performance Level

This describes how fast, smooth, and efficient the app is.

Levels:

- High Performance → Native apps
- Medium Performance → Cross-platform & hybrid apps
- Low Performance → Web apps

Performance depends on:

- Code efficiency
- Access to hardware

- Development method

1. Types of Mobile Applications

In Cameroon, your choice depends heavily on your budget and the "reach" you need across different phone types (low-end Transsion devices vs. iPhones).

1. Native Mobile Applications

Definition

Native applications are developed specifically for a single operating system, such as Android or iOS.

Technologies Used

- Android: Java, Kotlin
- iOS: Swift, Objective-C

Characteristics

- Installed through official app stores (Google Play Store, Apple App Store)
- Full access to device hardware such as:
 - Camera
 - GPS
 - Microphone
 - Bluetooth

Advantages

- High performance and speed
- Excellent user experience (UI matches platform standards)
- Full access to all device features

Disadvantages

- High development cost (separate apps required for each platform)
- Longer development time

Examples

- WhatsApp
- TikTok
- Uber

2. Web Applications

Definition

Web applications are mobile-friendly applications that run in a web browser and do not require installation.

Technologies Used

- HTML
- CSS
- JavaScript

Characteristics

- Accessed via a URL
- No installation required
- Compatible with all devices with a browser

Advantages

- Low development cost
- Easy to maintain (single codebase)
- Platform-independent

Disadvantages

- Requires internet connection

- Limited access to device hardware
- Lower performance compared to native apps

Examples

- Mobile banking websites
- Online shopping websites

3. Hybrid Applications

Definition

Hybrid applications combine elements of both web and native applications. They are essentially web apps wrapped inside a native container.

Technologies Used

- HTML, CSS, JavaScript
- Frameworks: Ionic, Apache Cordova

Characteristics

- Installed like native apps
- Use WebView to display content

Advantages

- Single codebase for multiple platforms
- Faster and cheaper development

Disadvantages

- Lower performance than native apps
- User interface may not feel fully native

Examples

- Early versions of Instagram

- Some e-commerce applications

4. Cross-Platform Applications

Definition

Cross-platform applications are developed using a single codebase and can run on multiple operating systems.

Technologies Used

- **Dart (Flutter)**
- **JavaScript (React Native)**
- **C# (Xamarin)**

Characteristics

- Code is shared across platforms
- Uses native components for better performance

Advantages

- Reduced development time
- Lower cost compared to native apps
- Near-native performance

Disadvantages

- Slightly lower performance than fully native apps
- Possible platform-specific issues

Framework Providers

- Flutter
- React Native
- Xamarin

5. Progressive Web Applications (PWA)

Definition

Progressive Web Apps are web applications that provide a mobile app-like experience.

Features

- Can be installed on the home screen
- Works offline
- Fast loading speed

Advantages

- No need for app store distribution
- Lightweight and efficient

Disadvantages

- Limited access to device features
- Not fully supported on all platforms

Overall conclusion

Type	Performance	Cost	Device Access
Native	High	High	Full
Web	Low	Low	Limited
Hybrid	Medium	Medium	Partial

Cross-platform	High	Medium	Almost Full
PWA	Medium	Low	Limited

2. Mobile Apps Programming Languages

Introduction

Programming languages are used to develop mobile applications by writing instructions that define how the application behaves and functions.

2.1. Android Programming Languages

Java

- One of the oldest Android programming languages
- Object-oriented and widely used

Advantages

- Large developer community
- Extensive libraries and tools

Disadvantages

- Requires more code (verbose)
- Slower development compared to modern languages

Kotlin (Modern Standard)

- Official Android development language

Advantages

- Concise and readable code
- Improved safety (reduces errors)
- Supports modern programming features

Disadvantages

- Slight learning curve for beginners

2.2. iOS Programming Languages

Swift

- Modern language for iOS development

Advantages

- Fast and efficient
- Easy to read and write

- Safe and less error-prone

Disadvantages

- Limited to Apple ecosystem

Objective-C

- Older programming language for iOS

Characteristics

- Still used in maintaining older applications
- More complex syntax compared to Swift

2.3. Cross-Platform Programming Languages

JavaScript

- Used in React Native

Advantages

- Very popular and widely used
- Easy to learn

Dart

- Used in Flutter

Advantages

- High performance
- Excellent for building user interfaces

C+

- Used in Xamarin

Advantages

- Strong support for enterprise-level applications
- Integrated with Microsoft tools

2.4. Other Supporting Languages

SQL

- Used for managing and querying databases

HTML & CSS

- Used in web and hybrid applications for interface design

2.5 Comparison Table

Language	Platform	Difficulty	Usage
Java	Android	Medium	Traditional apps

Kotlin	Android	Easy	Modern apps
Swift	iOS	Easy	Apple applications
JavaScript	Cross-platform	Easy	React Native
Dart	Cross-platform	Medium	Flutter
C++	Cross-platform	Medium	Xamarin

Key Points

- Kotlin is the modern standard for Android development
- Swift is the primary language for iOS
- JavaScript is used in React Native
- Dart is used in Flutter
- Native applications use platform-specific languages

Conclusion

Understanding the types of mobile applications and the programming languages used to develop them is essential for building efficient, scalable, and user-friendly mobile solutions. Each type and language has its strengths and limitations, and the choice depends on project requirements, cost, performance needs, and target audience.

3. Mobile App Development Frameworks

Mobile app frameworks are the foundational structures that developers use to build mobile applications. They provide a standard way to build and deploy applications and are integral in simplifying the development process.

Mobile apps development used to be a time-consuming affair, with so many different platforms in the market. In the late 2000s, these were just some of the mobile platforms that developers must develop for: iOS, Android, Symbian, BlackBerry, Windows Mobile/Phone, Palm, Java ME.

3.1 Types of mobile development frameworks

Mobile Apps Development can be broadly classified into two categories:

- Native app development
- Cross-platform app development

A native app is an application that is developed for a specific platform, such as iOS or Android. These apps are designed to run on a particular device or OS and can take full advantage of the platform's features and capabilities. Native apps are optimized for the platform and have access to the platform's native APIs. As such, native apps are more efficient, have better security, and have UI that adheres to the design guidelines and standards of the platform.

Cross platform on the other hand, aims to minimize the need to maintain separate code bases for each target platform. The mantra of cross platform development is – *Write Once, Run Everywhere*.

3.2 Some tools and frameworks of mobile development

3.2.1 Native app tools

XCode

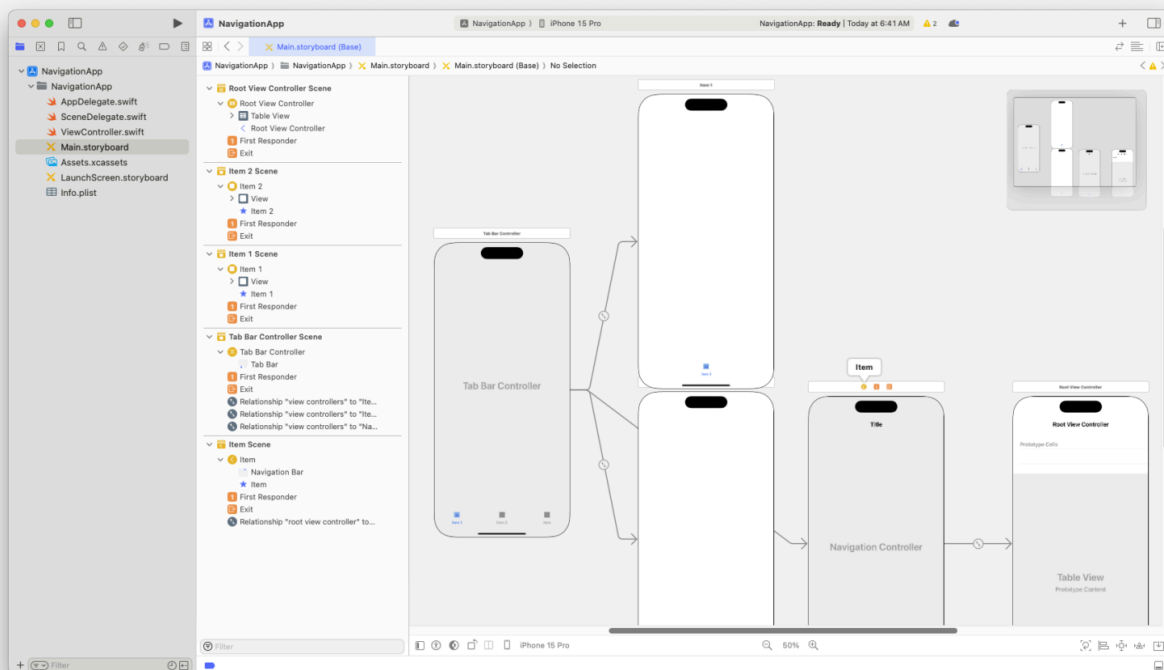
Xcode is an integrated development environment (IDE) created by Apple for developing software applications on its various platforms, including iOS, macOS, watchOS, and tvOS. It is the official and most widely-used development tool for Apple ecosystem development.

To develop iOS apps, you can use the Objective-C or the Swift programming language. For new projects, it is recommended that you start with Swift due to its modern language constructs, improved performance, and enhanced code readability. While Swift has become the primary language for Apple platform development, Objective-C codebases are still being maintained due to the use of legacy code and third-party libraries. Hence, expect to see Objective-C continue to have a presence in the Apple app ecosystem for years to come.

Xcode provides two approaches to building your apps' user interfaces:

- Storyboard
- SwiftUI

Storyboard is a visual interface design tool that allows developers to build the UI of their applications by arranging and connecting components and screens. Storyboards are useful when you need to have an overview of your app's flow and is a very useful prototyping tool. The following figure shows Storyboard in Xcode:



Android Studio

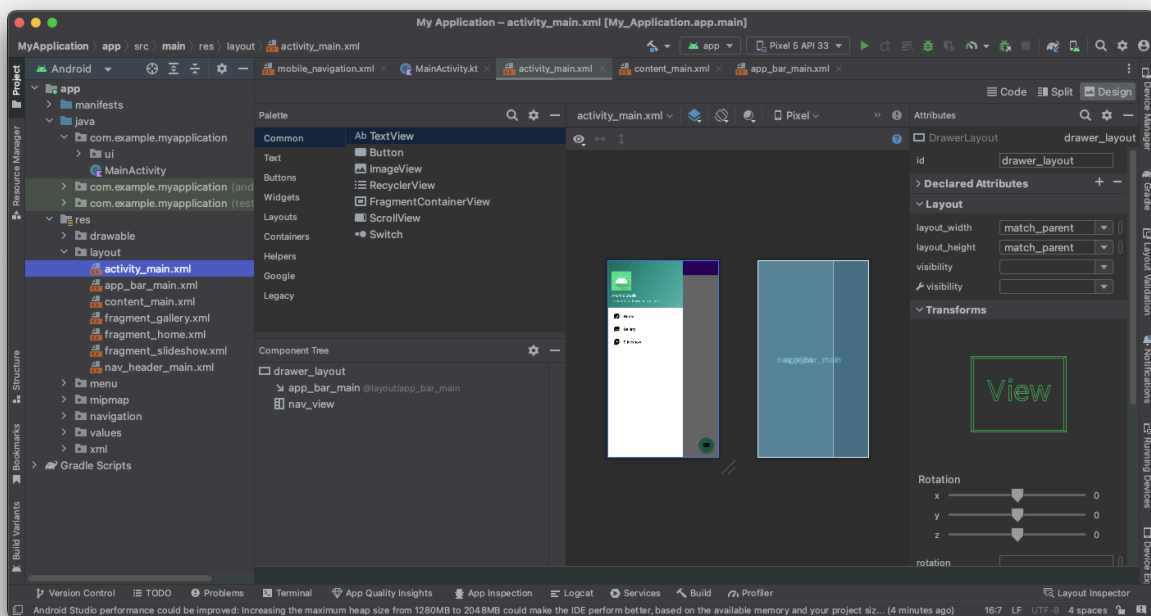
Android Studio is the official integrated development environment (IDE) for Android app development. It is a powerful and feature-rich IDE provided by Google to make it easier for developers to create, test, and publish Android applications.

For Android development, you can use the following programming languages:

- Java
- Kotlin

Kotlin is a modern, statically-typed programming language that runs on the Java Virtual Machine (JVM). It was developed by JetBrains and officially endorsed by Google for Android app development. Kotlin is fully compatible with Java and can work with existing libraries or frameworks written in Java. For new projects, the recommended language is Kotlin.

For building UI, Android Studio comes with the Layout Editor, where you can build your Android UI visually using drag-and-drop:



Android Studio comes with the Device Manager which allows you to test your applications using various Android emulators.

3.2.2 Cross platforms frameworks

React Native

React Native was first introduced by FaceBook (now called Meta) in 2015 as a way to build natively rendering mobile applications using the popular React JavaScript library. It is an

open-source UI framework created for developers to write apps that run on the following platforms:

iOS, macOS, tvOS, Android, AndroidTV, Web, Windows, and UWP (Universal Windows Platform)

The key advantage of React Native is you can maintain a single code base to target multiple platforms.

React Native works almost identical to React, except that React Native does not manipulate the DOM via Virtual DOM.

React Native apps are written using a mixture of:

JS (JavaScript), CSS (Cascading Style Sheet), JSX (JavaScript XML)

In particular, JSX allows developers to write HTML-like code within their JavaScript code, making it easier to create and manipulate the UI of their applications.

During compilation, React Native uses a tool called Metro to bundle all your code, dependencies, assets, and bundles into a single file, and then *transpiles* (translate-and-compile) your JSX code into ES5. They are then used to generate platform specific code for the platforms you are targeting. So, when you define a <Button> in React Native, the button will translate to platform-specific button styles on iOS and Android to maintain a consistent look and feel on both platforms.

To develop React Native mobile apps, you need the following:

- Node.js
- Xcode (requires a Mac)
- Android Studio

Flutter

Another cross-platform mobile development framework that is gaining a lot of popularity is Flutter. Like React Native, Flutter is a mobile framework for building natively compiled applications for mobile, web, and desktop from a single codebase. Flutter is created by Google, and uses Google's Dart programming language.

Flutter has the following major components:

- Flutter engine – written in C++, provides low-levels rendering support using
- Google's Skia graphics library Foundation Library – written in Dart, provides a base layer of functionality for apps and APIs to communicate with the engine

- Widgets – basic building blocks for UI

Unlike React Native, Flutter does not use the platform's native widgets. Instead, Flutter provides a set of widgets (including Material Design and Cupertino (iOS) widgets), managed and rendered by Flutter's framework and engine. Using this approach, it is much easier to maintain UI consistencies for your applications across platforms.

To develop Flutter mobile apps, you need the following:

- Flutter SDK
- Xcode (requires a Mac)
- Android SDK, Android Studio (optional)

3.2.3 Comparative analysis of react native and flutter

Aspect	React Native	Flutter
Developed by	Meta	Google
Programming Language	JavaScript (with React)	Dart
When to Use	- When you already know JavaScript - For apps needing quick development - When sharing code with web apps	- When you want high performance - For custom UI designs - When building from scratch
Performance	Good (uses native components)	Very high (compiled to native code)
UI Design	Uses native UI components	Uses its own widgets (more control)
Learning Curve	Easier (if you know JS)	Slightly harder (Dart is less common)

**Community
Support**

Very large and mature

Growing fast but smaller

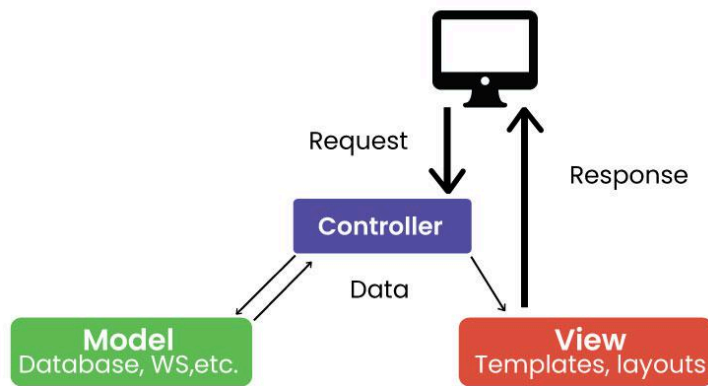
4. Mobile App Architectures and Design Patterns

Design patterns are reusable solutions to common software development problems. They have had a significant impact on software development, including mobile app development. The implementation of mobile apps has established some proven models and standards to overcome the challenges and limitations of mobile app development.

Most mobile applications were built with low code and were not based on architecture. Mobile app development with the right design patterns can effectively integrate user interfaces with data models and business logic. This will affect the quality of your source code.

4.1 Model View Controller (MVC) Architecture

MVC is a design model that separates an application into three interacting parts: Model, View, and Controller. This separation allows for better code design and modularization.



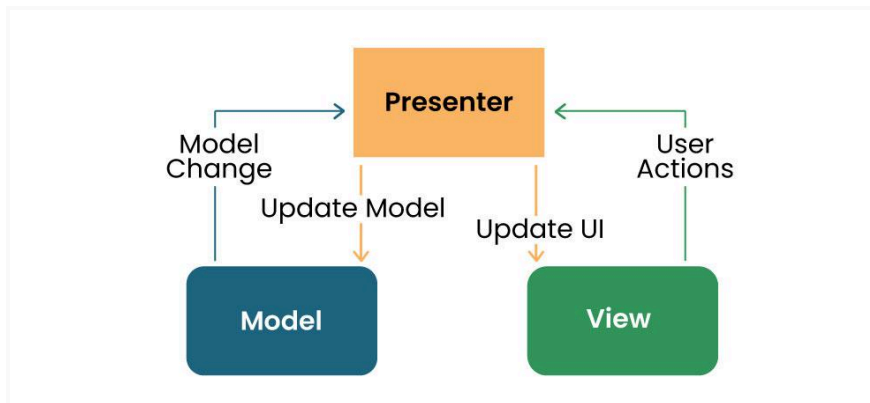
- **Model:** Represents application data and business logic.
- **View:** Displays data to the user.
- **Controller:** Processes user input and controls data flow between Model and View.

For Example:

Imagine a mobile weather app. The model stores weather information, the View displays it to the user, and the controller handles user interactions such as updating the displayed location or converting units (e.g. from Celsius to Fahrenheit).

4.2 Model View Presenter (MVP) Architecture

MVP is a new architecture that separates an application into three parts: Model, View, and Presenter. This is similar to MVC but puts more responsibility on the Teacher to manage the interaction between Model and View.



- **Model:** Manages data and business logic.

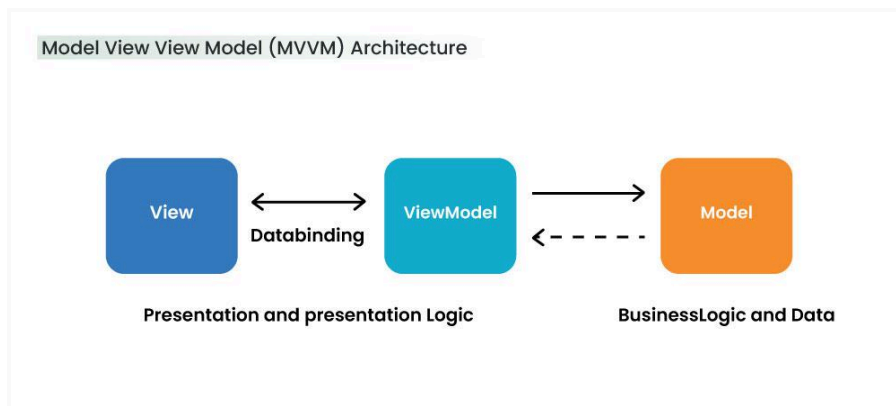
- **View:** Represents the user interface.
- **Designer:** Acts as an intermediary processing user input and updating the View and Model.

For Example:

In a note-taking app, the Model would store the text, the View would display it, and the provider would handle user input such as typing, editing, or deletes the process.

4.3 Model View View Model (MVVM) Architecture

MVVM is a design model widely used in mobile development, especially in frameworks like Android's Jetpack. Its purpose is to separate the application into three parts: Model, View, and ViewModel.



- **Model:** Represents data and business logic.
- **View:** Represents the user interface.
- **ViewModel:** Acts as an interface between the Model and the View, which contains the reference logic.

For Example:

In an e-commerce application, the Model contains product data, the View displays product information, and the ViewModel manages interactions, such as adding items to a cart.

4.4 Pros and Cons of Mobile Development Architecture Patterns

Model-View-Controller (MVC)

Pros:

1. **Separation of Concerns:** MVC clearly separates the application into three interconnected components, making it easier to manage and scale.
2. **Ease of Understanding:** The simplicity of MVC makes it straightforward to learn and implement, especially for beginners.
3. **Reusability:** Components within MVC can be reused across different parts of the application or even in different projects.
4. **Parallel Development:** Different team members can work on the Model, View, and Controller simultaneously, speeding up development.

Cons

1. **Complexity in Larger Applications:** For complex applications, the tight coupling between components can lead to difficulties in managing and scaling the codebase.
2. **Maintenance Challenges:** Changes in one component can require updates in others, increasing the maintenance effort.
3. **View-Controller Coupling:** In practice, the View and Controller can become tightly coupled, making it harder to test and maintain.

Model-View-ViewModel (MVVM)

Pros:

1. **Separation of Concerns:** MVVM promotes a clear separation between the UI and business logic, improving maintainability.
2. **Testability:** The ViewModel, being independent of the View, is easier to test.
3. **Data Binding:** Many MVVM frameworks support data binding, which can simplify the synchronization between the UI and the underlying data.
4. **Reusable Components:** ViewModels can be reused across different views, enhancing code reuse.

Cons:

1. **Learning Curve:** MVVM can be more complex to understand and implement compared to MVC, especially for beginners.
2. **Boilerplate Code:** The separation can sometimes lead to additional boilerplate code, making the initial setup more cumbersome.
3. **Performance Overhead:** Data binding can introduce performance overhead, especially in large applications with complex UIs.

5. Requirement Engineering Process

Requirement Engineering (RE) is the systematic process of defining, documenting, and maintaining the requirements for a mobile application, ensuring the final product aligns with user needs and business goals.

The Requirement Engineering process for mobile apps is unique because it must account for diverse hardware, varying screen sizes, and specific platform guidelines (iOS/Android). It typically follows these five key stages:

1. Requirements Elicitation

This is the "discovery" phase where you gather information from stakeholders, potential users, and market research.

- **Stakeholder Interviews:** Talking to business owners to understand the core problem the app solves.
- **User Research:** Surveys or focus groups to identify what features users actually want (e.g., offline mode, biometric login).
- **Market Analysis:** Reviewing competitors to find functional gaps your app can fill.

2. Requirements Analysis and Negotiation

In this stage, you sift through the raw data gathered during elicitation to check for conflicts, technical feasibility, and priority.

- **Conflict Resolution:** If one stakeholder wants a complex 3D interface and another wants it to work on low-end devices, you must find a middle ground.
- **Feasibility Study:** Determining if the requested features can be built within the budget and timeline using tools like Flutter, Swift, or Kotlin.
- **Prioritization:** Distinguishing between "Must-Haves" (MVP features) and "Nice-to-Haves."

3. Requirements Specification

This involves documenting the requirements in a clear, formal manner that both developers and stakeholders can understand.

- **Functional Requirements:** What the app *does* (e.g., "The app must allow users to track orders in real-time").
- **Non-Functional Requirements:** How the app *behaves* (e.g., "The app must load within 2 seconds" or "The app must consume minimal battery").
- **User Stories:** Writing requirements from the user's perspective: *"As a user, I want to filter food by price so I can stay within my budget."*

4. Requirements Validation

Before writing a single line of code, you must ensure the documented requirements are accurate and complete.

- **Prototyping:** Creating wireframes or low-fidelity mockups in tools like Figma to show stakeholders how the requirements translate to a UI.
- **Reviews/Walkthroughs:** A formal meeting where the team "reads through" the requirements to catch logical errors or missing edge cases.

5. Requirements Management

Requirements often change during development (Scope Creep). This phase handles those changes systematically.

- **Traceability:** Mapping each requirement to the specific code or test case to ensure nothing is missed.
- **Change Control:** A process for evaluating the impact of adding a new feature mid-sprint, ensuring it doesn't break the existing architecture or timeline.

6. Mobile App Development Cost Estimation (Cameroon Context)

Prices vary based on whether you hire a freelancer or an agency in cities like Douala, Yaoundé, or Buea.

App Complexity	Description	Estimated Price (FCFA)
Simple / MVP	Basic UI, 5-8 screens, local storage (e.g., a simple directory or brochure app).	300,000, - 500,000
Medium	Database, User Authentication, Payment Integration (Momo/OM), Admin Panel.	500,000 - 100,0000
Complex	Real-time tracking, AI, Complex Backend, High Security (e.g., a "Super App" or Bank app).	1,000,000 – infinity

1. Payment Gateways: Integration & Operational Leakage

When integrating local aggregators like **Monkap**, **Campay**, or **Faysal**, the cost is rarely just a "setup fee." You must evaluate the total cost of ownership (TCO) for financial transactions:

- **Tiered Transaction Fees:** Most local providers charge a percentage + a flat fee per transaction. For high-volume, low-ticket items, these fees can eat **3–5%** of your margin.
- **Settlement Latency:** Consider the "opportunity cost" of your cash flow. If an aggregator holds funds for T-2 or T-3 days, your operational liquidity is restricted.
- **Security & Compliance:** Budget for SSL certificates and potential PCI-DSS compliance audits if you plan to store or handle sensitive user payment data directly within your app's infrastructure.

2. Maintenance: The "Mobile OS Tax" & Technical Debt

The **15-20%** annual maintenance rule is a baseline, but mobile apps face unique pressures compared to web platforms:

- **Operating System Drift:** Every year, Apple (iOS) and Google (Android) release major updates. If your app uses specific plugins or hardware features (like GPS or Biometrics), you must budget for engineering hours to update libraries and fix breaking changes to remain listed on the stores.
- **App Store Maintenance:** There are recurring costs for developer accounts (\$99/year for Apple) and the labor required to manage "App Store Optimization" (ASO) updates, such as new screenshots and localized descriptions to keep conversion high.
- **Bug Fixing & User Support:** Post-launch, the first 6 months usually require intensive "hotfixing" as you encounter different hardware configurations and screen densities that weren't caught in initial QA.

3. Infrastructure: Cloud Scalability & Currency Volatility

Infrastructure costs for platforms like **AWS**, **Google Cloud**, or **Firebase** are dynamic and carry significant external risks:

- **The Exchange Rate "Burn":** Since these services are billed in **USD** or **EUR**, a sudden 10% devaluation of the local currency (XAF) effectively increases your server bill by 10% overnight, even if your traffic remains the same. It is vital to maintain a "Currency Reserve" or hedge your budget.
 - **Egress and Storage Costs:** Many developers budget for "Compute" (running the code) but forget "Egress" (the cost of data leaving the server to the user's phone). If your app serves high-resolution images or media, data transfer fees can quickly become your largest line item.
 - **Environment Parity:** You aren't just paying for the "Live" app. You need to budget for Staging and Development environments to test new features safely, which can add **30-50%** to your total monthly cloud bill.
-

4. Third-Party API & Service Dependency

- **Usage-Based Pricing:** If your mobile app relies on Map services (like Google Maps SDK), SMS OTPs for login, or AI processing, these are often billed per request.
- **The "Success Trap":** If your app goes viral, these API costs can spike faster than your revenue model can adapt, leading to a high "burn rate" during periods of rapid user acquisition.

Conclusion:

- **Architecture Over Code:** Choosing the right pattern (MVVM/MVP) is the foundation for building scalable, easy-to-maintain applications.
- **The Cross-Platform Edge:** Frameworks like Flutter and React Native provide the ideal balance of performance and cost-efficiency for our local market.
- **Precision in Planning:** Strong Requirement Engineering prevents technical debt and ensures the final product effectively solves the user's problem.